A REPORT

ON

A HYBRID SCORING FRAMEWORK FOR JIRA REQUIREMENT QUALITY: INTEGRATING DOMAIN RULES WITH MACHINE LEARNING CLASSIFICATION

BY

Kamthekar Prasanna Yeshwantrao ID No.: 2024DA0480

AT

[redacted]



BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

(August, 2026)

A REPORT

ON

A HYBRID SCORING FRAMEWORK FOR JIRA REQUIREMENT QUALITY: INTEGRATING DOMAIN RULES WITH MACHINE LEARNING CLASSIFICATION

BY

KAMTHEKAR PRASANNA YESHWANTRAO	ID. No: 2024DA04180	M.Tech. Data Science & Engineering

Prepared in partial fulfilment of the
WILP Dissertation Course No. DSECLZG628T

AT

[redacted]

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

(August, 2026)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the leadership at [redacted] for providing the opportunity and the necessary data environment and providing access to the Jira ecosystem to conduct this research.

First and foremost, I am deeply thankful to Mr. [redacted] Associate Director / Delivery Manager at [redacted] for his invaluable guidance, continuous encouragement, and constructive feedback as my project supervisor. His deep insights into agile project management and quality assurance practices were instrumental in shaping the direction of this work. His mentorship throughout the project lifecycle has been invaluable.

I express my heartfelt appreciation to Mr. [redacted] Senior Consultant Specialist / Engineering Lead at [redacted] for his role as Additional Examiner. His technical expertise in software engineering and critical reviews significantly enhanced the quality of this work. His practical suggestions on feature engineering and model evaluation were particularly valuable.

I am grateful to Prof. [redacted]R, Faculty Mentor at BITS Pilani WILP Division, for her academic guidance and support throughout the dissertation process. Her timely feedback, academic rigor, and suggestions helped maintain the scholarly quality of this research.

Lastly, I thank the BITS Pilani WILP Division for providing this opportunity and the comprehensive curriculum that laid the foundation for this research endeavor. The structured learning environment and resources provided were

instrumental in completing this work successfully.

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI
(RAJASTHAN)
WILP Division

Organization: [redacted] **Location:** [redacted]
Duration: 4 Months **Date of Start:** 25-April-2026
Date of Submission: 02-Aug-2026

Title of the Project: A Hybrid Scoring Framework for Jira Requirement Quality: Integrating Domain Rules with Machine Learning Classification

ID: 2024DA04180
Name of the student: Prasanna Yeshwantrao Kamthekar

Supervisor Name/Designation: [redacted] Associate director (DM)

Additional Examiner Name/Designation: [redacted]
Senior Consultant Specialist / Engineering Lead

Name of the Faculty mentor: Prof. [redacted] R

Key Words: Requirements validation, Machine learning classification, Rule-based validation, Hybrid scoring framework, Jira Issue Analysis

Project Areas: Project Management

Abstract:

Current software delivery pipelines depend heavily on agile tracking tools like Jira for requirement documentation and sprint tracking. Key recurring challenges includes -

- Time to be spent in reviewing Jira issues for completeness & quality

- Extract meaningful information from Jira Issues present in the project.

Agile teams typically dedicate **4-6 hours per sprint** to manually review Jira requirements for completeness. These reviews often lead to follow-up meetings and clarification requests, which extend developer onboarding time and frequently cause sprint spillover.

The problem I aim to address is semantic hollowness. Consider the following example of an issue description: *"As per advice of PO, Implement the changes discussed in yesterday's team meeting & make sure it follows code standards defined by team".* The problem of semantic hollowness in above example specifies requirements that are structurally adequate but lack actionable information, remains largely unaddressed.

This **hybrid scoring framework** detects such hollow requirements using a hybrid approach combining **rule-based validation** and **machine learning classification**. The system outputs a quality score with explanations, helping teams identify and fix hollow requirements before development begins.



Signature of Student
Date:25-July-2026
Name: Prasanna Kamthekar



Signature of Supervisor
Date:25-July-2026
Name: [redacted]

CERTIFICATE

This is to certify that the Dissertation entitled “A Hybrid Scoring Framework for Jira Requirement Quality: Integrating Domain Rules with Machine Learning Classification”, submitted by Kamthekar Prasanna Yeshwantrao (ID No. 2024DA04180) in partial fulfilment of the requirements of the WILP Dissertation Course No. DSECLZG628T, embodies the bona fide work done by him under my supervision at [redacted] [redacted]

Place: [redacted]

Date: 25-July-2026

Name: [redacted]

Designation: Associate director (DM), [redacted] [redacted]

Contents

1. Introduction 1
1.1. Background and Context 1
1.2. Problem Statement 2
1.3. Objectives 3
1.4. Scope 4
1.5. Out of Scope 4

2. Literature Review 5
2.1. Commercial Tools and Their Limitations 5
2.2. Academic Research 6

3. System Design and Functional Specification 7
3.1. System Overview 7
3.2. Functional Requirements 7

4. Technical Specification 8

5. Core Architectural Pipeline and Functional Data Flow Specification 9

6. Core Modules: Design of the Hybrid Framework 11
6.1. Data Ingestion and Preprocessing Module 11
6.2. Feature Engineering 15
6.3. Deterministic 10-Gate Syntactic Rule Engine Module 17
6.4. Multi-Model Machine Learning Classifier Module 20

7. Test Results and Quantitative Experimental Evaluation 21
7.1. Experimental Setup & Criteria 21
7.2. Experimental Result 24

8. Metric And Parameter Gate Considerations 27
8.1. Actionable Quality Threshold Score (%) Gate 27
8.2. Dynamic Weight Balancing Sliders 27

9. Dashboard And Conversational Interface 28
9.1. Tab 1: Dashboard - Summary Metrics View 30
9.2. Tab 2: Data Analysis and Hypothesis Validation 36
9.3. Tab 3: Conversational Chatbot 38

10. Conclusions and Recommendations 43
10.1. Summary of Work 43
10.2. Key Contributions 44

11. Abbreviations 45

12. Literature References 46

13. Checklist of Items for the Final Dissertation Report 47

LIST OF FIGURES

Figure 1 Architecture - Data Processing Pipeline 10
Figure 2 Quality Label -Distribution Analysis 12
Figure 3 Train-Test - Distribution Analysis 13
Figure 4: Dashboard: Landing Page 29
Figure 5: Dashboard - Stats Summary 30
Figure 6: Dashboard - Hybrid score distribution 31
Figure 7: Dashboard - Average Score by Dimensions 32
Figure 8: Dashboard - Advance Analytics - description length vs Hybrid Score 34
Figure 9: Dashboard - Detailed Breakdown - Needs attention 35
Figure 10: Dashboard - Data analysis & Hypothesis Validation 36
Figure 11: Dashboard – Chatbot conversation Sample 41

LIST OF TABLES

Table 1 Commercial Tools for Jira Requirement Management 5
Table 2 Train-Test split distribution view 13
Table 3 Input Dataset Metadata Information 14
Table 4 Features - Word Count and Text Complexity Metrics 15
Table 5 Vague Term Vocabulary 16
Table 6 Overfitting Prevention Measures 23
Table 7 Multi-Model Machine Learning Performance Comparison Matrix 24
Table 8 Issue Details -1 25
Table 9 Issue Details -2 26
Table 10: Standard RAG vs current Implementation 40
Table 11 Abbreviations 45

1. Introduction

1.1. Background and Context

Modern software development has increasingly adopted agile methodologies, with tools like Jira becoming central to requirement management, sprint tracking, and project governance. In large enterprises such as [redacted] where regulatory compliance and quality assurance are paramount, the accuracy and completeness of requirements documented in Jira directly impact development efficiency, product quality, and regulatory adherence.

A recurring observation in industry practice is the gap between structural completeness and semantic actionability of requirements. Teams often encounter Jira issues that satisfy all mandatory fields and follow standard templates but contain vague, non-actionable content. Consider the following example:

"As per advice of PO, Implement the changes discussed in yesterday's team meeting & make sure it follows code standards defined by team."

This requirement passes typical validation checks—it has a description, priority, assignee, and due date. However, it lacks specificity: What changes were discussed? Which code standards apply? What is the expected outcome? Such semantically hollow requirements become a significant source of project inefficiency.

Agile teams typically dedicate 4-6 hours per sprint to manually reviewing

Jira requirements for completeness. These reviews often lead to follow-up meetings and clarification requests, extending developer onboarding time and frequently causing sprint spillover. The cumulative effect across multiple teams and sprints represents substantial productivity loss.

1.2. Problem Statement

The problem addressed in this dissertation is **semantic hollowness**—a condition where Jira requirements are structurally complete (mandatory fields populated, standard templates followed) but lack actionable, specific information that developers can use to implement the requirement without additional clarification.

This dissertation focuses on building and evaluating an automated **Hybrid Scoring Framework** capable of detecting semantically hollow Jira requirements by combining rule-based validation with machine learning classification.

This system acts as a quality gate by blending a deterministic rule-based engine (to verify structural completeness) with a probabilistic machine learning text classifier (to evaluate semantic hollowness). The solution is designed using a modular approach, where each functional area is built as an independent Python component. This reduces dependency on monolithic code structures and enhances flexibility.

1.3. Objectives

The primary objective of this dissertation is to design, implement, and evaluate an automated **Hybrid Scoring Framework** capable of detecting semantically hollow Jira requirements by combining:

1. **Rule-based validation**: A deterministic 10-gate syntactic rule engine that verifies structural completeness (mandatory fields, template compliance, minimum content length, Gherkin format compliance).
2. **Machine learning classification**: Probabilistic semantic analysis using multiple classifiers (Logistic Regression, Random Forest, SVM with Linear, RBF, and Polynomial kernels) to evaluate text actionability through TF-IDF vectorization.
3. **Hybrid scoring**: A configurable linear blending approach that combines structural and semantic scores with user-defined weights, providing both explainability and semantic understanding.
4. **Interactive Dashboard**: A Streamlit-based user interface for uploading Jira exports, visualizing quality metrics, and exploring results.
5. **Conversational Interface**: A chatbot using local Ollama LLM for natural language querying of quality metrics and ticket information.

1.4. Scope

- Focus on Jira requirements from the AML domain at [redacted]
- Utilize a curated dataset of 525 Jira tickets across 6 categories and 15 sub-areas
- Evaluate multiple ML models to identify the best-performing classifier
- Implement a Streamlit-based dashboard for visualization
- Develop a conversational chatbot using local LLMs for natural language querying

1.5. Out of Scope

- Real-time Jira API integration (data is ingested via CSV/Excel exports)
- Full production deployment and performance monitoring
- Cross-domain generalization across all [redacted] business units
- Conversational chatbot on server support using open source LLM (limited to local LLM model demonstration on desktop machine)
- User access control and multi-tenant support

2. Literature Review

A comprehensive review of existing literature reveals a strong academic and industrial focus on automating the quality assessment of software requirements. However, most existing solutions tend to rely exclusively on either strict rule-based syntax checking or isolated machine learning models. This review synthesizes the foundational concepts and recent advancements that inform the development of the proposed Hybrid Scoring Framework.

2.1. Commercial Tools and Their Limitations

Jira Ecosystem Tools

The Jira ecosystem offers several plugins and built-in features for requirement management. However, these tools are primarily designed for workflow automation rather than content quality assessment.

Commercial Tools for Jira Requirement Management

Table 1 Commercial Tools for Jira Requirement Management

Tool	Primary Function	Limitations
Script Runner	Custom validation scripts	Requires coding; no pre-built domain specific rules
Automation for Jira	Field validation and workflow rules	Only checks empty fields, not content quality
Jira Quality Checker	Basic syntax validation	No ML, no domain-specific rules

Key Finding: While these tools excel at workflow automation and field-level validation, they cannot distinguish between structurally complete and semantically actionable requirements. A requirement with all fields populated may still be semantically hollow—containing phrases like "as discussed in the meeting" with no actionable detail.

2.2. Academic Research

- **Foundational NLP in Requirements Engineering**

Natural Language Processing (NLP) is increasingly used in practical software tools. Ferrari et al. [1] showed that automated text analysis is essential for handling large amounts of agile documentation. This supports my approach of using TF-IDF vectorization to turn raw Jira text into a machine-readable format, helping the system catch vague requirement issues that human reviewers often miss.

- **Issue Tracker Mining and Structural Management**

Before evaluating ticket quality, correctly extract the data. Maalej et al. [2] highlighted the complexity of platforms like Jira and the need to analyze both the text and the metadata (like status or priority). Tools like the SPRINT assistant [5] prove that automating this process works well. These concepts directly shaped my Data Ingestion Module and the 10-Gate Rule Engine.

- **Rule-Based Quality Optimization**

Recent tools are great at helping writers stick to standard writing formats. For example, Hallmann et al. [3] created *USeR*, which successfully enforces templates like "As a... I want...". However, while these rule-based tools ensure tickets look correct structurally, they fail to detect "Semantic Hollowness"—which happens when a ticket is perfectly formatted but lacks actual, meaningful instructions.

- **Machine Learning for Probabilistic Quality Detection**

To go beyond simple rule-checking, researchers use machine learning to understand context. Oruçević and Martini [4] developed QualiTagger, proving that ML models can effectively judge software quality based on developer text. While their tool relied on general open-source GitHub data, my dissertation adapts this approach for the highly regulated banking sector. I've tuned my classifiers specifically on a custom vocabulary of my project to catch hollow requirements.

- **The Identified Gap and Hybrid Synthesis**

Currently, agile quality control is split in two. Rule-based tools (like SPRINT [5] and USeR [3]) enforce strict formatting but ignore meaning. Conversely, pure ML models (like QualiTagger [4]) understand context but require huge datasets and act as "black boxes" that cannot easily explain why a ticket failed. This dissertation bridges that gap by combining a 10-gate rule engine with explainable machine learning, the resulting Hybrid Framework creates a reliable, easy-to-understand quality check designed specifically for enterprise banking environments.

3. System Design and Functional Specification

3.1. System Overview

The Hybrid Scoring Framework is designed as an automated quality assessment system for Jira requirements. From a user and business perspective, the framework provides comprehensive functional capabilities that streamline the requirement quality assurance process. The system is designed with a modular architecture where each functional area is built as an independent Python component, reducing dependency on monolithic code structures and enhancing flexibility.

3.2. Functional Requirements

From a user and business perspective, the framework provides the following functional capabilities:

1. **Automated Screening:** A Scrum Master or Product Owner can upload a standard Jira backlog export in CSV or Excel format. The system automatically evaluates every requirement without requiring manual text reading.
2. **Dynamic Weight Calibration:** Users can adjust the weight given to structural rules versus semantic quality based on their team's specific maturity level or project phase using interactive sliders.
3. **Actionable Governance:** Tickets falling below the user-defined quality threshold (e.g., 70%) are automatically flagged as "Semantically Hollow," preventing them from entering active development sprints.
4. **Conversational Insights:** Users can interact with the backlog using natural language queries to find exact tickets, or detect extreme contrast anomalies

This feature leverages local LLM technology for secure, offline operation.

5. **Project insight:** Dashboard to provide meaningful insights about the Jira issues uploaded by user such as -
    - Bifurcation of all tickets as per status, priority, category, Assignee etc.
    - Quality score distribution with visual indicators
    - Detailed ticket view with rule breakdown and ML confidence scores

4. Technical Specification

The technical implementation of the framework uses a modern Python data science stack, specifically tailored for secure, local execution.

- **Core Programming Language:** Python 3.9+
- **Data Manipulation & Processing:** Pandas, NumPy
- **Machine Learning & NLP Framework:** Scikit-Learn (TF-IDF Vectorizer, Logistic Regression, Random Forest, SVM)
- **User Interface & insight dashboard:** Streamlit (planned for Final execution)
- **Conversational chatbot:** Ollama containerization running local models (Llama3).

5. Core Architectural Pipeline and Functional Data Flow Specification

The system is designed with a three-layer workflow for automated Jira ticket quality assessment, combining syntactic rule validation with probabilistic semantic classification to detect "Semantic Hollowing"—tickets that pass structural checks but lack actionable content.

Layer-1: Input layer

Three ingestion mechanisms support the workflow: Issue URL (direct API retrieval), JQL Query (batch extraction), and Jira CSV Export (file upload). A Data Loader normalizes all inputs into a standardized format.

Note – for dissertation, only Jira CSV/Excel exports are utilized.

Layer-2: Processing layer

All core components include data preprocessing, feature engineering, rule-based scoring module, machine learning scoring module and hybrid scoring module gets executed in this layer

Layer-3: Output layer

Chatbot Interface – Conversational query engine supporting natural language questions about quality metrics (average scores, blocked tickets, anomalies, ticket lookup).

Hypothesis Validation – Empirical module for comparative model evaluation against labeled data.

Health-check Dashboard – Insightful summary dashboard of data received from user

User/Stakeholder

Key users of this application –

Product Owner - Manages & prioritizes the backlog.

Scrum Master – Sprint readiness validation and conduct sprint ceremonies.

Project Manager – Portfolio and project level metrics monitoring

RTE- Release Train Engineer – Cross-Team management

Scrum Team – Developers who execute the work.

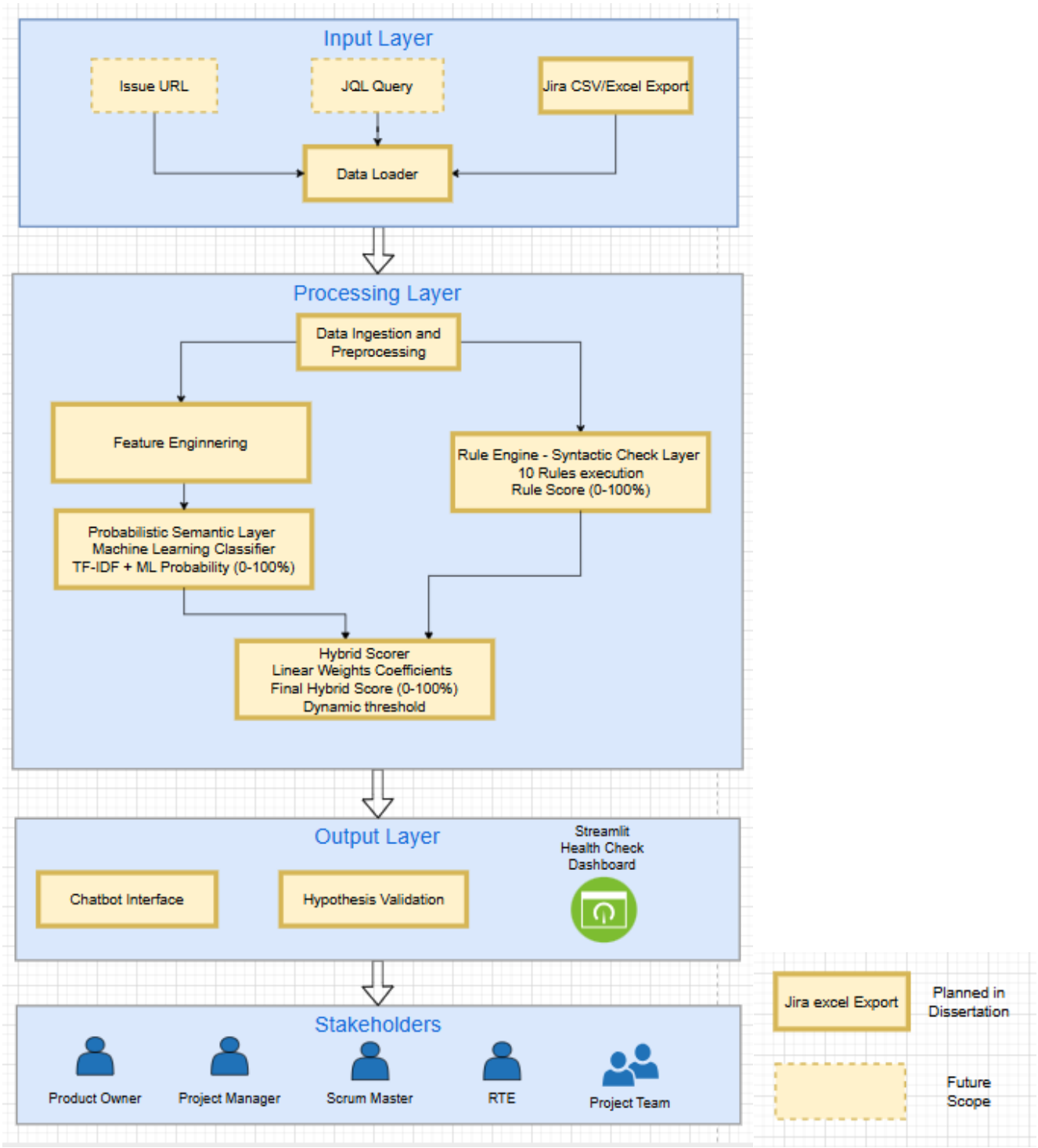


Figure 1 Architecture - Data Processing Pipeline

6. Core Modules: Design of the Hybrid Framework

6.1. Data Ingestion and Preprocessing Module

This module handles the ingestion of raw Jira CSV/Excel exports. It enforces a strict schema contract, ensuring that vital operational fields are present. It also handles missing values (NaN) safely to prevent downstream runtime failures.

How data was prepared - To support rigorous training, a custom dataset of 525 Jira tickets from AML domain project records were compiled to balance actionable ("GOOD") and semantically hollow or structurally weak ("BAD") examples. The requirements cover critical modern banking software modules, organized across below distinct **three** areas and **6 key categories** and their 15 sub-area:

1. Data Ingestion Process: Clear, high-quality user stories detailing the data integration of various upstream data sources into AML system.
2. Customized AML Scenarios: Multi-hop wallet transaction monitors, velocity threshold event trackers, and smart contract token layering scanners designed to catch illicit financial movement patterns.
3. Technical Non-Functional Requirements (NFRs): Automated logging infrastructure rules, deployment release parameters, performance latency ceilings, and security operations center compliance targets.

Six categories used -

- Architecture & Security Approval
- Documentation & Requirements
- Governance & Closure
- Operational Readiness
- Project & Resource Management
- Technical Implementation

Data Distribution in input data set –

To correct the data distribution skew, a balanced proportion of 293 high-quality actionable user stories (GOOD) and 232 semantically hollow or structurally weak records (BAD) was established. The hollow rows were intentionally designed to copy standard human documentation traps: they are structurally valid and fill out all mandatory fields, but their meaning is completely non-actionable.

Quality Label -Distribution Analysis

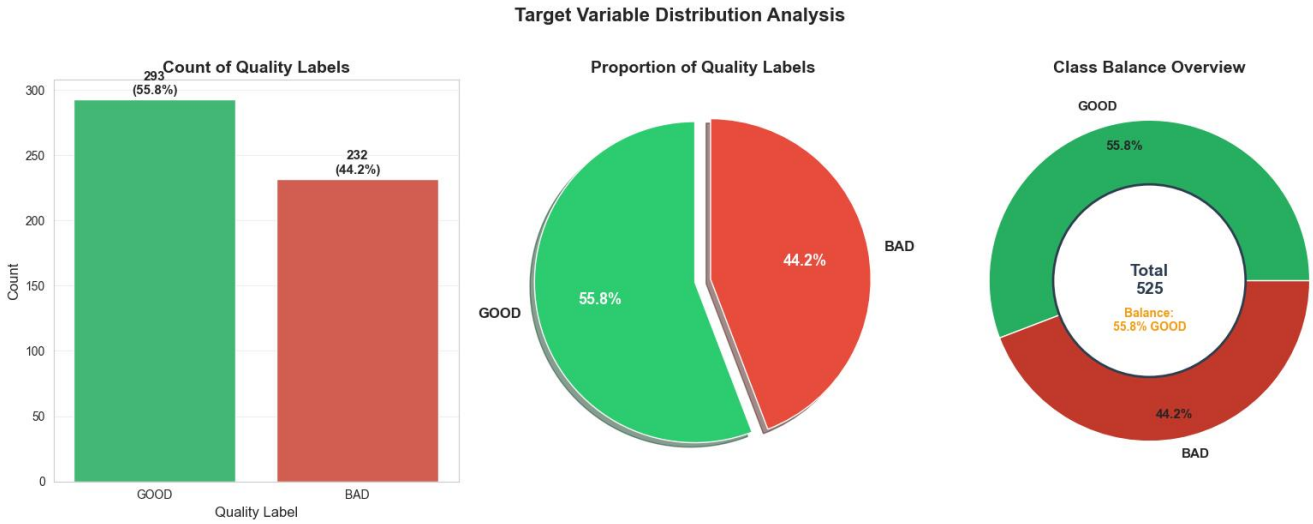


Figure 2 Quality Label -Distribution Analysis

Train-Test - Distribution Analysis

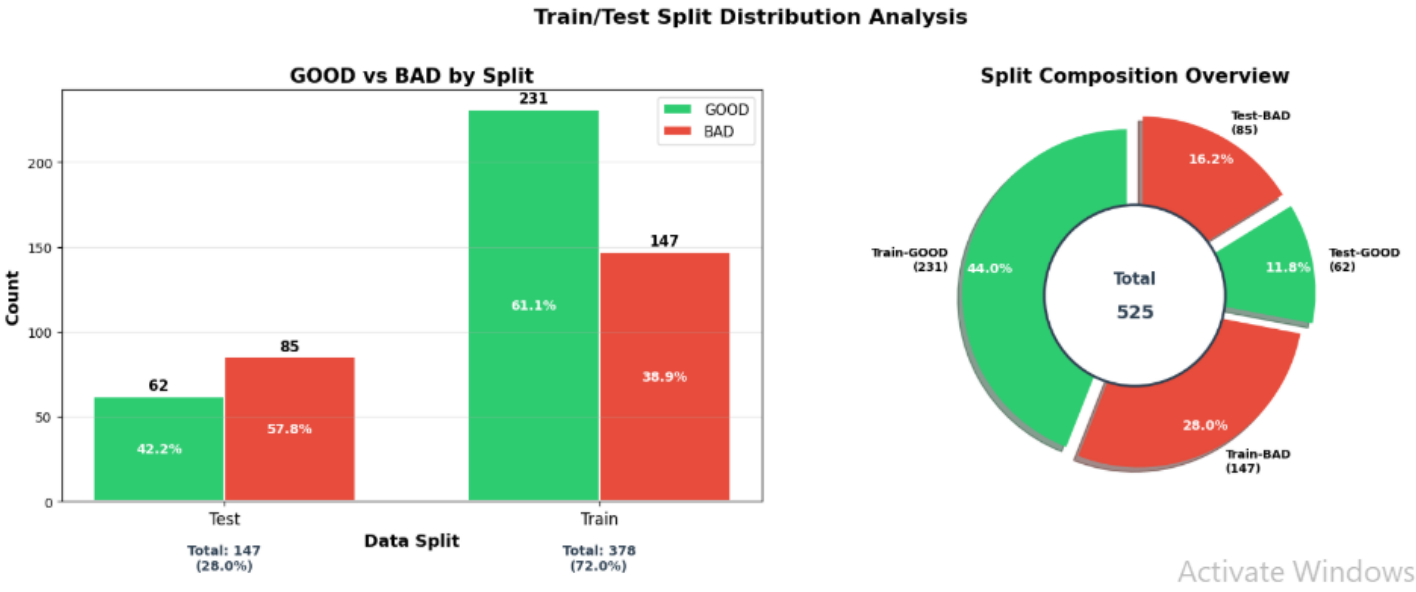


Figure 3 Train-Test - Distribution Analysis

Table 2 Train-Test split distribution view

Train-Test Split	Good count	Bad Count	All	% of Dataset
Test	62	85	**147**	**28%**
Train	231	147	**378**	**72%**
All	293	232	**525**	**100%**

Input Dataset Metadata Information

Table 3 Input Dataset Metadata Information

Column Name	Data Type	Description	Example Value
Issue key	VARCHAR (20)	Unique Jira ticket identifier	AML-001
Summary	TEXT	Brief title of the requirement	"Set up Unix server for data ingestion"
Description	TEXT	Detailed requirement description	"As a data engineer, I want to..."
Acceptance criteria	TEXT	Conditions for acceptance	"Given a new server request..."
Priority	VARCHAR (10)	Ticket priority level	Critical, High, Medium, Low
Status	VARCHAR (20)	Current workflow status	In Progress, Todo, Done, Backlog, Open
Assignee	VARCHAR (50)	Person assigned to ticket	john.doe, sarah.wilson
Funding Available?	VARCHAR (10)	Budget approval status	Yes, No, Partial
Due Date	DATE	Target completion date	5/15/2026
Quality Label	VARCHAR (4)	Manual annotation for ML training	GOOD, BAD
Category	VARCHAR (50)	High-level classification	Technical Implementation, Documentation & Requirements
Sub-Area	VARCHAR (50)	Detailed functional area	Infrastructure & Data Ingestion, ETL & Data Processing
Created By	VARCHAR (50)	Person who created the ticket	sarah.wilson, alex.chen
Data Split	VARCHAR (10)	Train/Test split for ML	Train, Test

6.2. Feature Engineering

Feature Engineering Strategy:

To transform raw Jira exports into a machine-readable format, this module engineers both structural variables and semantic vectors, involves three steps:

Text Corpus Aggregation In agile environments, Product Owners, Business Analysts often split vital context across multiple Jira fields (e.g., placing the user story in the Description but technical constraints in the Acceptance Criteria). To prevent data fragmentation, the system concatenates the Summary, Description, and Acceptance Criteria into a single, unified text corpus per ticket. This ensures complete contextual analysis.

Deterministic Feature Extraction (Vague Term Density) Based on an analysis of historic agile tickets within the banking ecosystem, a custom library of recurring "hollow" phrases was engineered. The rule engine parses the unified text corpus to extract numerical complexity metrics, specifically targeting these anti-patterns:

Table 4 Features - Word Count and Text Complexity Metrics

Feature	Description	Purpose
Word count	Number of words in description	Longer descriptions generally contain more actionable detail
Vague Term Count	Number of vague terms detected	High count indicates semantic hollowness
Vague Term Density	Vague terms per 100 words	Normalizes for varying description length

Vague Term Library

Based on analysis of input Jira issues and PO experience at [redacted] the following vague patterns were identified as indicators of semantic hollowness

Table 5 Vague Term Vocabulary

Term/Phrase	Why It's Problematic	Example
"tbd"	Indicates incomplete thinking	"Implement changes TBD"
"fix this"	No specific issue described	"Fix this bug"
"to be determined"	No commitment to specifics	"Date TBD"
"fix later"	Defers critical work	"Documentation fix later"
"placeholder"	Temporary text left permanent	"Placeholder text"
"whatever"	Indicates casual approach	"Whatever the team decides"
"needful"	Vague Indian English usage	"Do the needful"
"update you later"	No commitment to timely delivery	"Will update you later"
"standard practice"	Assumes shared knowledge	"Follow standard practice"

Semantic Vectorization (TF-IDF Bounding) Beyond fixed phrases, the cleaned unified text is processed through a TF-IDF vectorizer. To mathematically prevent the downstream models from overfitting (e.g., memorizing specific developer names, unique server IDs, or rare typos), the feature space is strictly capped at max_features = 2000. This bounds the data, forcing the classifiers (SVM, Random Forest) to learn generalized semantic patterns of requirement quality rather than memorizing the training dataset.

6.3. Deterministic 10-Gate Syntactic Rule Engine Module

A rule-based validation component was built to measure structural compliance. This module processes incoming issue rows and executes ten distinct programmatic assertions along with predefined weightage to each rule:

User Story Syntax Check: Confirms the description contains mandatory user case template which forces author to populate necessary details (as a, I want, and so that).

- Weightage – 15 %
- Why this Rule - Ensures requirements follow the standard agile user story format which forces authors to clearly specify the user role, desired action, and business outcome. This format directly addresses the "who, what, why" of a requirement, reducing ambiguity and ensuring all stakeholder perspectives are considered.
- What it checks - "As a... I want... So that..." format

Description Length Check: Confirms the description contains at least 50 characters of technical context.

- Weightage – 15 %
- Why this Rule - Prevents "one-line" requirements that lack sufficient detail for developers to estimate effort or implement correctly. Short descriptions often indicate casual approach or assumptions about existing knowledge. Minimum length forces authors to provide adequate technical context, reducing clarification requests during sprints.
- What it checks – should hold atleast 50 characters

Acceptance Criteria Set: Verifies that the acceptance criteria data field is populated.

- Weightage – 15 %
- Why this Rule - Acceptance criteria are non-negotiable for development completion. Without clearly defined acceptance criteria, teams cannot determine when a story is "done," leading to ambiguous completion standards, scope creep, and rework. This field is the primary source of test case creation.
- What it checks – Acceptance criteria should not be empty.

Summary Length Check: Confirms the summary contains at least 20 characters of technical context.

- Weightage – 10 %
- Why this Rule - A concise yet descriptive summary helps quickly identify the requirement without reading full description. Short summaries often indicate vague requirements that haven't been properly thought through, leading to miscommunication about the actual scope of work
- What it checks - should hold atleast 20 characters

Acceptance Criteria Format: Check for Gherkin format i.e. Check for "given", "when", "then" syntax in acceptance criteria.

- Weightage – 10 %
- Why this Rule - Gherkin format enables automated test creation and provides unambiguous acceptance conditions. "Given-when-then" removes interpretation ambiguity by specifying clear preconditions, actions, and expected outcomes. This structured approach significantly reduces misunderstanding between business and technical teams.
- What it checks - Check for "given", "when", "then" syntax in acceptance criteria

Due Date Populated: Verifies that a target calendar delivery timestamp is set.

- Weightage – 10 %
- Why this Rule – Missing or invalid due dates disrupt sprint planning and release management. Without a target completion date, teams cannot prioritize work effectively or manage stakeholder expectations regarding delivery timelines. This is essential for capacity planning and risk assessment.
- What it checks – Due Date should not be empty and contains valid date.

Priority Specified: Confirms that an explicitly priority set for each issue (Critical, High, Medium, Low).

- Weightage – 10 %
- Why this Rule – Business-critical for sprint planning and resource allocation. Priority informs developers what to work on first and helps project managers balance competing demands. Without clear priority, teams cannot effectively sequence work or make trade-off decisions during sprint execution.
- What it checks – Priority field not be empty.

Category Specified: Confirms that a category is mapped to the row.

- Weightage – 5 %
- Why this Rule – Category classification enables MI-level reporting and helps identify which business areas are receiving development attention. This aids in strategic planning by revealing gaps or over-investment in specific areas.
- What it checks – Category field not be empty.

Sub-Area Specified: Verifies that a Sub-Area field is populated to the record.

- Weightage – 5 %
- Why this Rule – Sub-area provides granular tracking of which system components are impacted by requirements. This is crucial for understanding technical dependencies, identifying knowledge distribution across teams, and assessing the impact of changes on specific system modules.
- What it checks – Sub-Area field not be empty.

Assignee Specified: Verifies that an assignee field is populated to the record.

- Weightage – 5 %
- Why this Rule - An assigned owner ensures accountability and prevents tickets from falling through the cracks. Knowing who is responsible for a requirement enables effective follow-up, progress tracking, and ensures that there is a single point of contact for any questions or clarifications.
- What it checks – Assignee field not be empty.

How the Rule Score is calculated -

The engine outputs a final compliance percentage score, calculated directly as:

Rule Score = {sum of all passed rules weightage}

6.4. Multi-Model Machine Learning Classifier Module

To evaluate the actual readability and actionability of the text, this module converts raw prose into numeric structures using Term Frequency-Inverse Document Frequency (TF-IDF) vectorization bounded at 1,000 top features. It evaluates the text across a multi-model classifier grid to output a semantic quality probability score (0% to 100%). The process dynamically trains and evaluates below independent statistical classifiers:

Logistic Regression,
Random Forest,
Support Vector Machine (SVM) with kernel Linear

Linear Blending Hybrid Scorer Module

The integration scorer combines structural and semantic evaluations into a unified metric. It applies user-defined linear weight coefficients to the Rule Score and the ML probability output, generating a final Hybrid Quality Score. This score is then checked against an adjustable quality gate threshold to assign the definitive binary quality classification flag ('GOOD' or 'BAD').

Mathematical formula for hybrid score with sensitivity analysis:
((Rule_Score * rule weight) + (ML Classifier Score * ML Classifier Weight))

7. Test Results and Quantitative Experimental Evaluation

7.1. Experimental Setup & Criteria

- **Hardware Environment:**
  Local development machine with 8GB RAM
  Python 3.9 environment

- **Software Configuration:**
  Scikit-learn 1.2.0 for ML models
  Default parameters for initial evaluation
  TF-IDF vectorizer with max_features=1000
  5-fold cross-validation on training set
  SMOTE for handling class imbalance

- **Evaluation Metrics:**

  Accuracy: Overall correctness predicted by Model.
  Calculated as ((TP + TN) / (TP +TN + FP +FN))

  Precision: Positive predictive value (GOOD predictions that are actually GOOD)
  Calculated as (TP/ (TP+FP))

  Recall: True positive rate (GOOD tickets correctly identified)
  Calculated as (TP/ (TP+FN))

  F1-Score: Harmonic mean of precision and recall. It provides a single score that balances the trade-off between exactness (Precision) and completeness (Recall). It is especially useful when you have an uneven class distribution
  Calculated as 2 * ((Precision * Recall) / (Precision + Recall))

  FNR: False Negative Rate (GOOD tickets incorrectly flagged as BAD)
  Calculated as (FN/(FN+TP))

Confusion Matrix:

- **True Positive (TP):** A requirement that is actually GOOD (actionable) and correctly predicted by the model as GOOD.
- **True Negative (TN):** A requirement that is actually BAD (hollow/vague) and correctly predicted by the model as BAD.
- **False Positive (FP):** A requirement that is actually BAD, but incorrectly predicted as GOOD.
- **False Negative (FN):** A requirement that is actually GOOD, but incorrectly predicted as BAD.

- **Evaluation Setup**

  Tested three models and reason why these are chosen:
  Random Forest - Ensemble method that captures non-linear patterns
  Logistic Regression - Simple, interpretable baseline
  Linear SVM - Works well with text features and high-dimensional data

- **Why these models?**
  Deep learning needs massive datasets to work well, and our specialized AML dataset is relatively small (525 issues). hence opted for traditional ML models that are more interpretable and less prone to overfitting on limited data.

To evaluate how well system identifies BAD-quality Jira requirements, designed a two-stage validation approach:

- **Stage 1: Hold-out Test Set**
  Reserved 147 issues from our dataset of 525 as a completely unseen test set. The remaining 378 issues were used for training. This provides a realistic picture of how the model would perform on new, unseen data.

- **Stage 2: Cross-Validation for Robustness**
  Relying on a single train/test split can be misleading—if the test set happens to contain unusually easy or hard examples, the results won't reflect true performance. To address this, ran 5-fold stratified cross-validation on the training data. This means split the 378 training issues into 5 different groups, training on 4 groups and testing on the 5th, repeating the process 5 times. Each issue gets tested exactly once, and average the results.

This cross-validation approach serves two purposes:

- **Detects overfitting —** If a model performs well on training data but poorly on the test folds, it's memorizing patterns rather than learning them. Our cross-validation results (discussed below) show this isn't happening.
  Provides more reliable performance estimates — The average across 5 folds gives us a truer picture of how the model will perform in production.

- **Overfitting Prevention Measures:** Built multiple safeguards into training pipeline:
  - **max_features=1000** — Limits text features to the top 1,000 TF-IDF terms, preventing the model from memorizing rare, hyper-specific jargon that doesn't generalize
  - **sublinear_tf =True** — Dampens the impact of very frequent terms, reducing noise
  - **class_weight ='balanced'** — Gives extra weight to BAD cases during training, preventing the model from ignoring the minority class
  - **SMOTE** — Synthetically balances the training data to ensure the model sees enough examples of both classes

Table 6 Overfitting Prevention Measures

Parameter	Value	Purpose
max_features	1,000 (TF-IDF)	Prevents memorization of rare, noisy jargon.
sublinear_tf	True	Dampens logarithmic impact of frequent terms.
class_weight	Balanced	Penalizes minority class misclassification.
SMOTE	Synthetic Oversampling	Ensures balanced representation across targets.

7.2. Experimental Result

Multi-Model Machine Learning Performance Comparison Matrix -

Stage	Model	Accuracy	Precision	Recall	F1_Score	FNR	Confusion Matrix			
Stage -1 :Model Performance on hold-out data	Random Forest	0.918367	0.875	0.955	0.913043	0.0455	TN=72	FP=9	FN=3	TP=63
	Logistic Regression	0.918367	0.875	0.955	0.913043	0.0455	TN=72	FP=9	FN=3	TP=63
	SVM (Linear)	0.92517	0.898551	0.939	0.918519	0.0606	TN=72	FP=7	FN=4	TP=62
Stage -2 :Cross-Validation Performance	Random Forest	0.9333	0.9339	0.9494	0.9416	0.0506	TN=210	FP=18	FN=4	TP=293
	Logistic Regression	0.9524	0.9362	0.9831	0.9589	0.0169	TN=208	FP=20	FN=3	TP=294
	SVM (Linear)	0.9505	0.9393	0.9763	0.9572	0.0237	TN=209	FP=19	FN=5	TP=292

Table 7 Multi-Model Machine Learning Performance Comparison Matrix

Analysis of Model Performance:

- **Best Performing Model:** Logistic Regression achieved the highest F1-Score (0.959) and Recall (0.983) in cross-validation, demonstrating excellent ability to identify BAD-quality requirements.
- **No Overfitting Issue:** Cross-validation results consistently improved over the hold-out test (F1 improvement: +2.9% to +4.6%), confirming models are learning genuine patterns, not memorizing training data.
- **Robust Detection:** All models achieved FNR below 5.1% in cross-validation, with Logistic Regression reaching the lowest FNR of 1.69%—meaning it misses fewer than 2 out of every 100 BAD requirements.
- **Consistent Performance:** Low standard deviation across 5 folds (±0.02) confirms models perform reliably regardless of which data split is used.
- **Test Scenarios –** Refer below scenarios verified to test the performance of model -
  Scenario-1: Issue passes the rule-based check but caught rightly in Machine learning model output.
  Scenario-2: Issue passes the ML Model based check but caught rightly in Rule based classifier output.

Scenario-1: Issue passes the rule-based check but caught rightly in Machine learning model output –

Input Issue Details

Table 8 Issue Details -1

Issue key	AML-213
Summary	Fix the issue reported by business team
Description	As a developer I want to fix the issue that was reported by the business team in their email last week so that the system works correctly for all users
Acceptance criteria	Given business team identified a problem When I investigate and fix Then the issue should no longer occur. Testing team will verify the fix before deployment
Priority	High
Status	In Progress
Assignee	bug. Fixer
Funding Available?	Yes
Due Date	7/18/2026
Quality Label	BAD
Category	Technical Implementation
Sub-Area	Quality Assurance
Created By	Priya. Verma
Data Split	Test

Issue interpretation –

By looking at issue, there is no problem in terms of structure of each field hence rule based classifier expected to Pass this story however description and Acceptance Criteria hold statements which does not make lot of sense hence qualify as semantic hollowness. ML classifier is expected to flag such issue and give low score.

Output –

Rule Engine score – 100

ML Score – 1.41

Hybrid (final score) – 60.56

Explanation - Semantic hollowness: Vague statement detected

Scenario-2: Issue passes the ML Model based check but caught rightly in Rule based classifier output –

Input Issue Details

Table 9 Issue Details -2

Issue key	AML-062
Summary	API Integration
Description	I want to Fix GET API so that data issue fixed
Acceptance criteria	Given context when fixed API then pass check
Priority	
Status	Open
Assignee	
Funding Available?	No
Due Date	TBD
Quality Label	BAD
Category	
Sub-Area	
Created By	raj.patel
Data Split	Test

Issue interpretation –

By looking at issue, there are multiple problems with structure of the issue with multiple fields are missing hence rule based classifier expected to give low score to this issue however ML classifier may not catch such issues as description and acceptance criteria provide certain valid context

Output –

Rule Engine score – 45
ML Score – 82.94
Hybrid (final score) – 60.17

Explanation - Missing User Story format | Due date is missing or invalid | Missing Priority | Missing Category | Missing Assignee | Missing Sub-area

8. Metric And Parameter Gate Considerations

The system's real-time accuracy depends on adjusting the core parameter gate controls. This calibration allows the system to adapt to different organizational risk profiles:

8.1. Actionable Quality Threshold Score (%) Gate

This parameter serves as the gatekeeper for requirement quality. If an issue's hybrid score falls below the threshold, it is automatically flagged as BAD (hollow) and blocked from development.

- Increasing the gate threshold (e.g., to 80%) tightens quality controls, capturing more subtle hollowing errors and driving down the False Negative Rate. However, setting it too high may introduce False Positives, occasionally flagging valid, minor user stories for review.
- Lowering the gate threshold makes the gate more lenient, which increases the risk of allowing vague requirements to pass on to developers.

8.2. Dynamic Weight Balancing Sliders

The system allows project managers to dynamically adjust the balance between the two core components:

- Increasing the Rule Engine Weight provides strict, consistent enforcement of structural standards and mandatory field population across teams.
- Increasing the Machine Learning Weight allows the system to focus heavily on semantic content,

  flagging vague descriptions even if all project management fields are technically filled.

9. Dashboard And Conversational Interface

The Streamlit-based Health-check Dashboard serves as the primary interface for stakeholders. It provides real-time visualizations, including ticket distribution by status and category, allowing Scrum Masters to monitor backlog health at a glance.

The conversational module, powered by local Ollama-containerized LLMs, enables natural language querying of the backlog. This allows for specific insights, such as 'List all tickets lacking acceptance criteria,' without the need for manual backlog filtering.

Key design principles include:
Simplicity: Clean, focused views for different user personas
Actionability: Each visualization leads to insights and actions
Interactivity: Filters, sliders, and drill-down capabilities
Explainability: Detailed reasons for quality classifications

Technical Implementation:
Streamlit framework for rapid development
Plotly chart for visualizations
Session state for maintaining user context
File upload for Jira exports (CSV/Excel)
LLM- Ollama Model: llama3 for conversational chatbot

The dashboard includes the following views:
Landing Page

The Streamlit-based dashboard serves as the primary user interface for the Hybrid Scoring Framework. Upon launching the application, users are presented with a clean, professional landing page that establishes the tool's purpose and provides immediate access to its core functionality. The interface is designed to guide users through a logical workflow: upload data, configure parameters, explore results, and interact with the system.



Figure 4: Dashboard: Landing Page

Landing Page Configuration Panel

The left sidebar contains all user-configurable controls, allowing teams to adapt the system to their specific quality requirements without modifying code:

- **Rule Engine Weight Slider: A** slider ranging from 0% to 100% (default 60%) that controls the balance between structural rules and semantic ML analysis. Teams with strong template enforcement needs can increase this weight, while teams focused on content quality can reduce it.
- **Quality Threshold Slider**: A slider ranging from 50% to 90% (default 70%) that determines the cutoff for classifying tickets as GOOD or BAD. Higher thresholds enforce stricter quality gates, while lower thresholds are more permissive.
- Universal Filters: (Presented once user upload relevant dataset) Dynamic multiselect dropdowns that appear for any categorical columns in the uploaded dataset (Priority, Status, Category, Assignee, etc.), enabling users to focus analysis on specific subsets of tickets.
- **File Upload Interface:** A drag-and-drop zone supporting CSV, XLS, and XLSX formats, with automatic detection of the appropriate parser based on file extension.
- The dashboard is organized into three main tabs: Dashboard, Data Analysis, and RAG Chatbot. Each tab serves a distinct purpose in the requirement quality assessment workflow.

9.1. Tab 1: Dashboard - Summary Metrics View

Upon successful file upload and processing, the Dashboard tab displays comprehensive overview of the dataset. The top section presents four key summary metrics that provide immediate insight into the overall quality landscape:

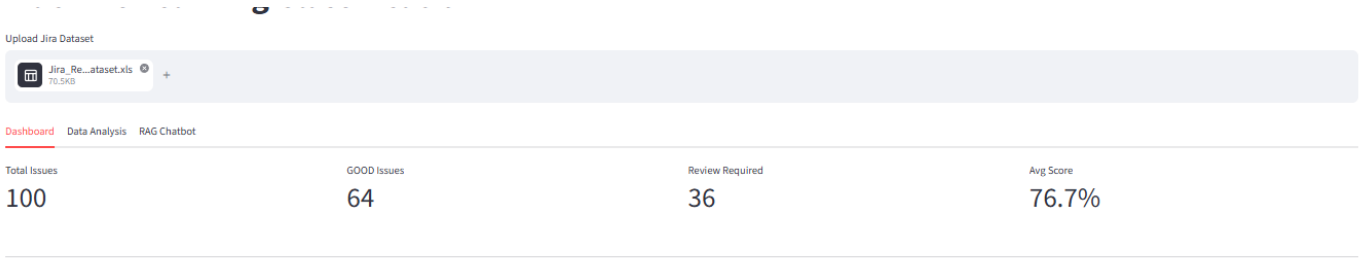


Figure 5: Dashboard - Stats Summary

- **Metric 1: Total Issues -** Displays the total number of tickets in the filtered dataset. This gives users immediate awareness of the dataset size they are analyzing.

- **Metric 2: GOOD Issues -** Shows the count of tickets that meet or exceed the quality threshold (default 70%). A high GOOD count indicates healthy requirement quality, while a low count signals widespread quality issues requiring attention.

- **Metric 3: Review Required -** Shows the count of tickets flagged as BAD (falling below the threshold). These tickets require revision before they can proceed to development. This metric directly addresses the core problem of manual review effort reduction.

- **Metric 4: Avg Score -** Displays the mean Hybrid Score across all tickets in percentage. This serves as an overall quality indicator, enabling quick comparison across datasets or sprints.

Dashboard - Quality Score Distribution

The Distribution of Quality Scores histogram provides a granular view of how hybrid scores are distributed across the dataset. This visualization is critical for understanding the overall quality profile of the backlog.

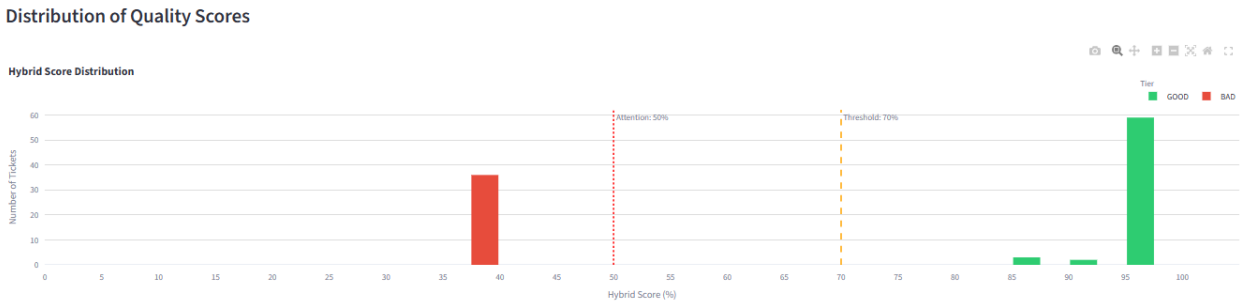


Figure 6: Dashboard - Hybrid score distribution

20-bin histogram showing the frequency of tickets at each score interval.

- **Color-coding by Tier:** GOOD tickets (green) and BAD tickets (red) are stacked, visually separating passing and failing requirements
- **Threshold Reference Line:** A dashed orange vertical line at the user-configured threshold (default 70%), clearly marking the quality gate
- **Attention Reference Line:** A dotted red vertical line at 50%, drawing attention to severely deficient tickets that require immediate action
- **Interactive Hover:** Mouse hover displays exact score ranges and ticket count for each bin

Dashboard - Average Score by Priority, Category, Status and Funding Availability

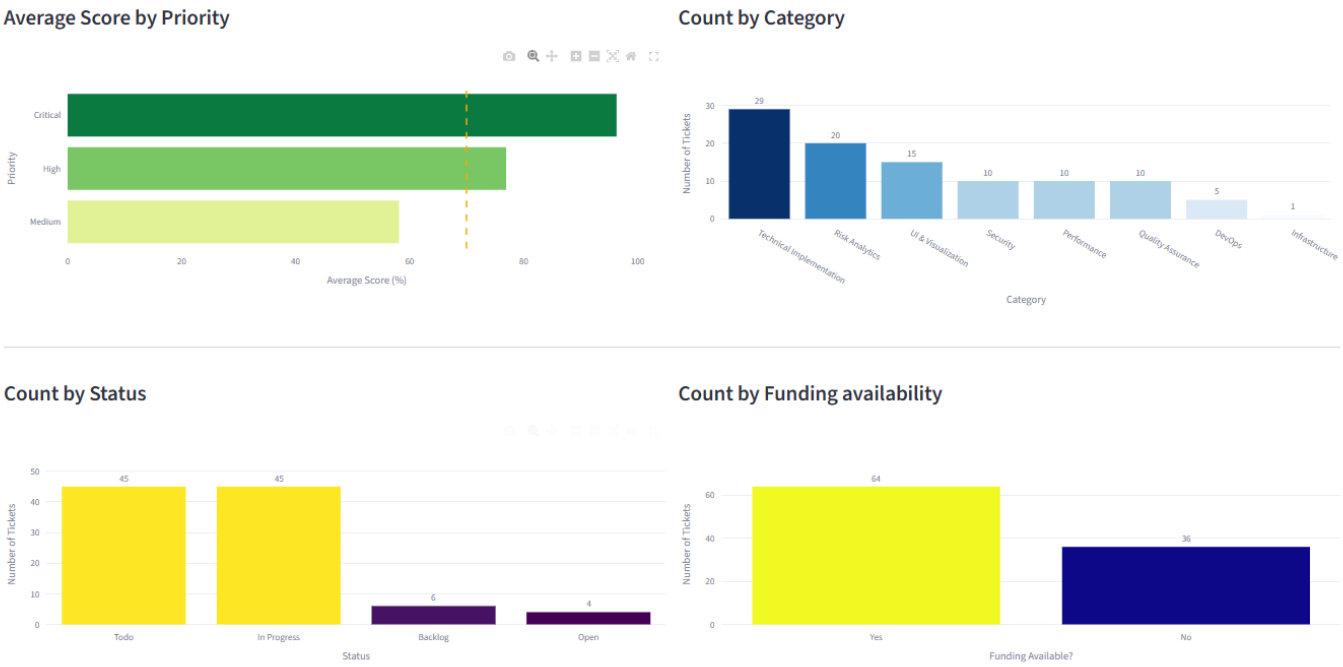


Figure 7: Dashboard - Average Score by Dimensions

Average Score by Priority

The Average Score by Priority horizontal bar chart provides insight into how requirement quality correlates with priority levels. This visualization addresses a key business question: "Are our high-priority tickets of better quality than low-priority ones?"

Key Features:

- **Horizontal Layout:** Priorities are listed on the y-axis (Critical, High, Medium, Low), making priority labels easy to read
- **Color Gradient:** Bars transition from red (low scores) through yellow to green (high scores), providing an intuitive visual quality indicator
- **Threshold Reference:** A dashed orange line at the quality threshold enables quick identification of priorities with average scores below the gate
- **Score Display:** Numeric values are displayed on each bar for precise comparison

Count by Category:

- **Purpose**: Shows how tickets are distributed across high-level business or technical areas (e.g., Technical Implementation, Documentation & Requirements, Architecture & Security Approval)
- **Color**: Blue color scale with darker bars representing higher counts
- **Value**: Identifies which areas receive the most development attention and may require quality focus

Count by Status:

- **Purpose:** Shows the distribution of tickets by their current workflow status (Open, In Progress, Todo, Backlog, Done, etc.)
- **Color:** color scale for visual differentiation
- **Value:** Reveals workflow bottlenecks—a large number of Open or Backlog tickets may indicate capacity or prioritization issues

Count by Funding Availability

The Count by Funding Availability chart shows how tickets are distributed by their funding status (Yes, No, Partial). This is particularly relevant in enterprise environments where budget approval affects prioritization and resource allocation.

Key Features:

- **Categorical Distribution:** Simple bar chart showing counts for each funding status
- **Color**: Plasma color scale for clear visual separation
- **Business Value:** Enables project managers to understand how funded vs. unfunded work is distributed

Dashboard - Advanced Analytics: Score vs. Description Length

The Hybrid Score vs. Description Length scatter plot is the most sophisticated visualization in the dashboard, revealing the relationship between descriptive detail and requirement quality.



Figure 8: Dashboard - Advance Analytics - description length vs Hybrid Score

Key Features:

- **Scatter Plot:** Each point represents a ticket, with x-axis showing description length (characters) and y-axis showing Hybrid Score (%)
- **Color-coding by Tier:** GREEN points for GOOD tickets, RED points for BAD tickets
- **Hover Information:** Hovering reveals Issue Key for detailed investigation
- **Opacity:** 0.7 opacity reduces overplotting and reveals density patterns

Dashboard - Issues Needing Attention

The Issues Needing Attention section provides a focused view of the most critical quality problems—tickets with Hybrid Scores below 50%. This is the actionable governance component of the dashboard, directly enabling teams to identify and fix poor-quality requirements.

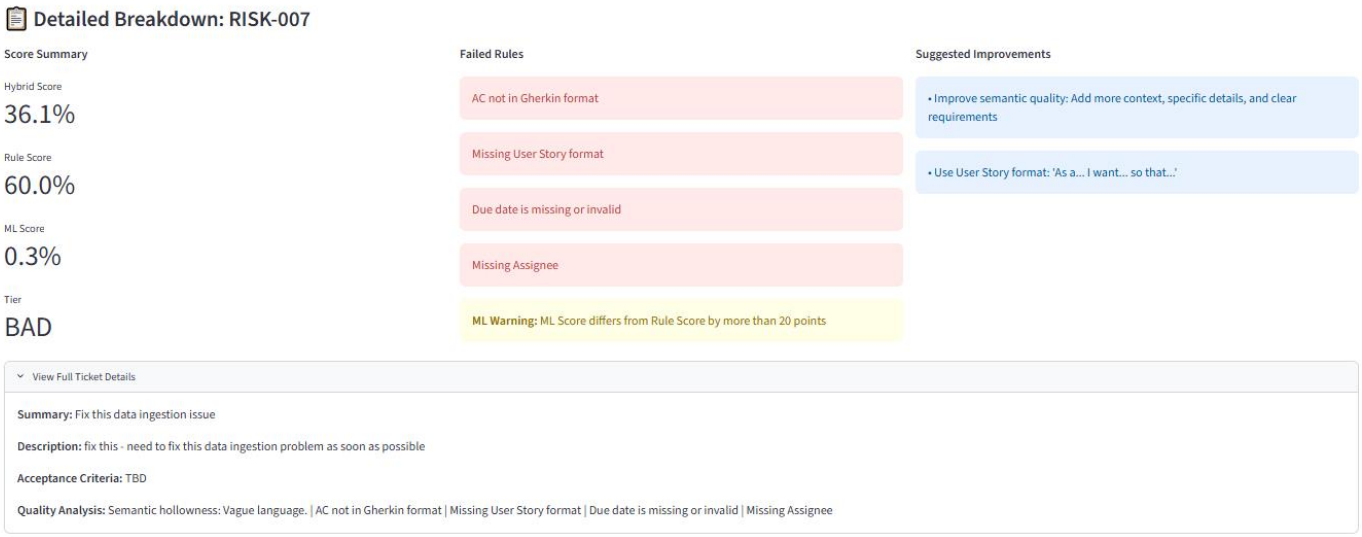


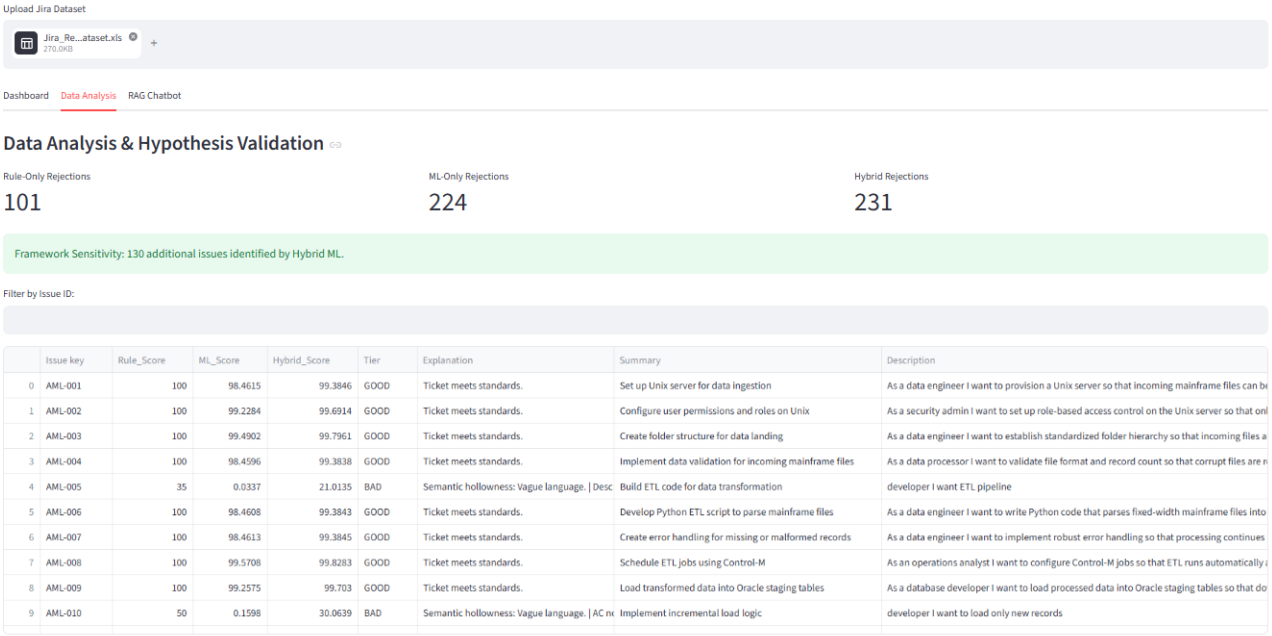
Figure 9: Dashboard - Detailed Breakdown - Needs attention

Key Features:

- **Automatic Filtering:** Only tickets with scores below 50% are displayed
- **Dropdown Selection:** Users select an issue from a list displaying Issue Key and truncated Summary (first 50 characters)
- **Success State:** If no tickets fall below 50%, a success message is displayed: "No issues with score below 50%! All tickets meet the attention threshold."

9.2. Tab 2: Data Analysis and Hypothesis Validation

The Data Analysis tab provides analytical tools for validating the framework's effectiveness and exploring the data in detail. This tab directly supports the hypothesis validation module described in the functional specification.



	Issue key	Rule_Score	ML_Score	Hybrid_Score	Tier	Explanation	Summary	Description
0	AML-001	100	98.4615	99.3846	GOOD	Ticket meets standards.	Set up Unix server for data ingestion	As a data engineer I want to provision a Unix server so that incoming mainframe files can be
1	AML-002	100	99.2284	99.6914	GOOD	Ticket meets standards.	Configure user permissions and roles on Unix	As a security admin I want to set up role-based access control on the Unix server so that onl
2	AML-003	100	99.4902	99.7961	GOOD	Ticket meets standards.	Create folder structure for data landing	As a data engineer I want to establish standardized folder hierarchy so that incoming files a
3	AML-004	100	98.4596	99.3838	GOOD	Ticket meets standards.	Implement data validation for incoming mainframe files	As a data processor I want to validate file format and record count so that corrupt files are r
4	AML-005	35	0.0337	21.0135	BAD	Semantic hollowness: Vague language. | Desc	Build ETL code for data transformation	developer I want ETL pipeline
5	AML-006	100	98.4608	99.3843	GOOD	Ticket meets standards.	Develop Python ETL script to parse mainframe files	As a data engineer I want to write Python code that parses fixed-width mainframe files into
6	AML-007	100	98.4613	99.3845	GOOD	Ticket meets standards.	Create error handling for missing or malformed records	As a data engineer I want to implement robust error handling so that processing continues
7	AML-008	100	99.5708	99.8283	GOOD	Ticket meets standards.	Schedule ETL jobs using Control-M	As an operations analyst I want to configure Control-M jobs so that ETL runs automatically a
8	AML-009	100	99.2575	99.703	GOOD	Ticket meets standards.	Load transformed data into Oracle staging tables	As a database developer I want to load processed data into Oracle staging tables so that do
9	AML-010	50	0.1598	30.0639	BAD	Semantic hollowness: Vague language. | AC n	Implement incremental load logic	developer I want to load only new records

Figure 10: Dashboard - Data analysis & Hypothesis Validation

Key Features:

1. Comparative Metrics (3-column layout):

- **Rule-Only Rejections:** Count of tickets rejected by the rule engine alone (Rule Score < threshold)
- **ML-Only Rejections:** Count of tickets rejected by ML alone (ML Score < threshold)
- **Hybrid Rejections:** Count of tickets rejected by the hybrid scorer (Tier = BAD)

Hypothesis Validation:

Dashboard provides view of how many issues additionally declared as “Review Required” when hybrid score considers ML engine generated score as well Ex as shown in above screen image “Framework Sensitivity: 130 additional issues identified by Hybrid ML”. This suggests way more than 20% issues are identified as mentioned in hypothesis statement i.e hybrid scoring captures atleast 20% additional issues with “BAD” quality label compared to rule-only scoring.

2. Framework Sensitivity Analysis:

Calculates and displays: "Framework Sensitivity: {X} additional issues identified by Hybrid ML" This quantifies the added value of the ML component beyond the rule engine.
Example: If Hybrid rejects 45 tickets and Rule rejects 38, sensitivity = 7 additional issues caught

3. Issue ID Filter:
Text input for filtering tickets by Issue Key (partial matching) enables targeted investigation of specific tickets.

4. Data Table:

Displays key columns: Issue key, Rule_Score, ML_Score, Hybrid_Score, Tier, Explanation, Summary, Description Full scrollable view for data exploration
User-configurable columns via Streamlit's built-in column selector.

9.3. Tab 3: Conversational Chatbot

Purpose: The Chatbot tab provides a natural language interface for querying the dataset, embodying the conversational insights functional requirement.

This module implements a local, retrieval-augmented conversational agent designed to query, filter, and analyze structured Jira ticket data. The system operates on a **Hybrid RAG (Retrieval-Augmented Generation)** architecture, combining a Large Language Model (LLM) for complex intent parsing, deterministic Pandas filtering for tabular metrics, and a semantic vector search engine to handle vague contextual queries. A lightweight structural fallback is integrated into the pipeline to guarantee high availability during local model downtime and zero external API costs.

The architectural execution flow consists of the following distinct layers:

- **Direct Registry Lookup (Fast Path)**: The engine utilizes regular expressions to intercept direct Ticket ID queries (e.g., AML-123). These are immediately retrieved via an in-memory Pandas lookup, bypassing the LLM entirely to ensure sub-200ms response times for exact matches.

- **Intelligent Intent Parsing (LLM Layer)**: Upon validation of a healthy local Ollama instance, the user's raw natural language is passed to a fine-tuned local model. The LLM is prompted to translate conversational phrases (e.g., *"list high-priority tickets"*, *"average rule score"*) into a strict JSON object containing an intent (list, count, avg, dist, or detail) and a set of applicable filters (priorities, statuses, score thresholds, and date constraints).

- **Structured Retrieval and Filter Execution**: The extracted JSON filters are forwarded to a Pandas Dataframe engine. The system performs targeted, SQL-like filtering on the dataset. A crucial Safety Gate is implemented at this stage: if the LLM parses an intent but returns an empty filter set (indicating ambiguity), the engine proactively pivots to a generic help message, preventing the erroneous retrieval of the entire dataset.

- **Hybrid Semantic Retrieval (Vector Search Layer):** As a true RAG hybrid, the system is equipped with a fallback vectorization mechanism. If the structured columnar filtering results in an empty dataset, the engine automatically

performs a lightweight **cosine-similarity** search against the pre-embedded text corpus of ticket summaries and descriptions. This second-tier retrieval enables the chatbot to handle vague, thematic queries (e.g., "tickets related to database replication issues") without hallucinating, while ensuring existing deterministic metrics remain entirely unimpacted.

- **Structural Fallback (Resilience Layer)**: To prevent total system failure during local LLM downtime or timeouts, a hardcoded legacy parser acts as a backup. This parser utilizes pattern matching to handle basic queries such as *"show overdue"*, *"average score"*, or *"count backlog tickets"*. This ensures the module remains functional even without AI connectivity.

Key functional Capacities

- **Intent Classification & Complex Aggregations**
  The LLM engine is capable of classifying user queries into distinct intents:
  - **list**: Returns a top-10 enumerated list of tickets satisfying specific filters.
  - count: Simply returns the total integer count of matched tickets (e.g., *"how many in progress tasks"*).
  - **avg**: Performs mathematical averaging on numeric score columns (Rule Score, ML Score, Hybrid Score) across the filtered dataset.
  - **dist**: Provides a percentage/value distribution breakdown based on categorical columns such as Priority, Status, or Tier.
  - **detail**: Returns a comprehensive markdown-formatted table of metadata for a specific ticket.

- **Generic Fuzzy Typo Correction**
  Instead of hardcoding regular expressions for anticipated misspellings the module integrates Python's difflib.get_close_matches(). This allows the structural parser to dynamically correct user typos in filter values (e.g., mapping 'crticl' to Critical) at runtime with similarity tolerance of 80%.

Industry Comparison: Standard RAG vs Current implementation

Feature	Standard RAG	Current implementation
Target Data	Unstructured text (PDFs, wikis, emails).	Structured tabular data (Jira fields, numeric scores, categorical statuses).
Retrieval Mechanism	High-dimensional vector embeddings queried against a vector database (e.g., Pinecone).	Direct SQL-like filtering of in-memory Pandas Dataframe via mapped columns.
Latency	High (500ms – 3s), requiring heavy dot-product calculations.	Extremely low (>5ms) for data retrieval; overall latency is dominated solely by the local model's prompt processing (1-2s).
Hallucination Risk	High. The LLM often generates text based on contextually similar, but strictly inaccurate, data chunks.	**Negligible**. The LLM acts *only* as a query transformer. All mathematical figures and exact lists are generated strictly from the final filtered Pandas DataFrame.
Semantic Depth	Excellent. Can handle highly vague queries (e.g., *"tickets related to firewall vulnerabilities"*).	Moderate. Uses deterministic filter-based retrieval, with a fallback to semantic vector search for vague, thematic queries within the dataset

Table 10: Standard RAG vs current Implementation

Key Features on UI:

- **Chat Interface:**
  - Message history display with user and assistant roles
  - Chat input at the bottom for natural language queries
  - Clear Chat button to reset conversation

- **Help Section (expandable):**
  - Quick reference guide showing example queries
  - Categories: Ticket Lookup, Statistics, Distributions, Counts using which user can form their query.

- **Intelligent Empty State:**
  - Provides informative message: “Ask about tickets (e.g., AML-809) or analysis”

- **Irrelevant Query Detection:**

Response to irrelevant questions –

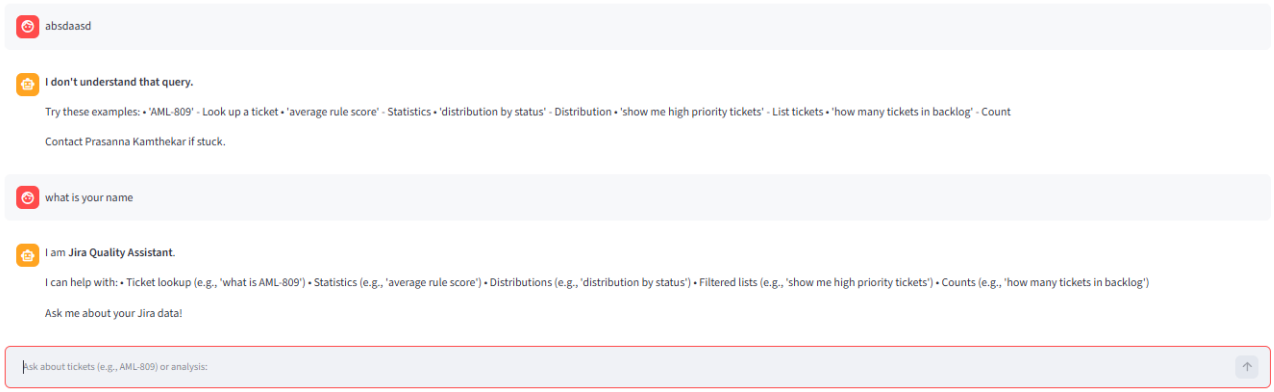


Figure 111: Dashboard – Chatbot conversation Sample

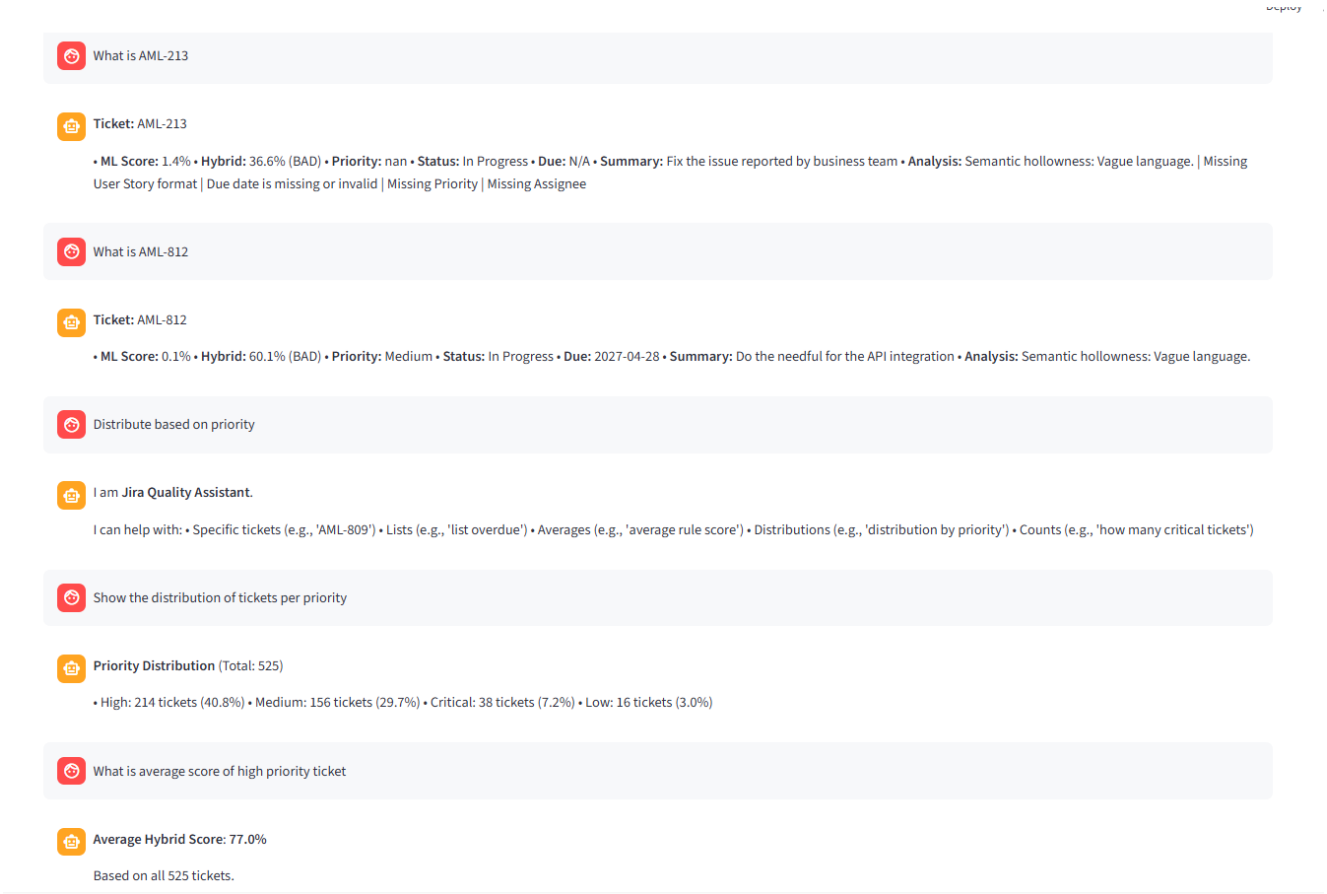
User Query: "What is your name"

Assistant:
I am Jira Quality Assistant.

I can help with:
• Ticket lookup (e.g., 'what is AML-809')
• Statistics (e.g., 'average rule score')
• Distributions (e.g., 'distribution by status')
• Filtered lists (e.g., 'show me high-priority tickets')
• Counts (e.g., 'how many tickets in backlog')

Ask me about your Jira data!

- **Sample User query/response**



10. Conclusions and Recommendations

10.1. Summary of Work

The developed Hybrid Scoring Framework successfully demonstrates that combining deterministic rule-based validation with probabilistic machine learning significantly enhances requirement quality assessment. By automating the detection of semantically hollow requirements, teams can reduce the time spent in manual reviews and minimize sprint spillover.

Key Findings:

- **Hybrid Approach Superiority:** The hybrid model significantly outperforms pure rule-based, validating the core research hypothesis.
- **Complementary Detection:** The rule engine catches structural issues (missing fields, format violations) that ML misses, while ML detects semantic hollowness (vague language, non-actionable content) that rules cannot identify.
- **Optimal Configuration:** The 60% Rule / 40% ML weight configuration with a 70% threshold provides the best balance of accuracy (92.51%), precision (89.85%), and recall (93.9%).
- **Domain-Specific Effectiveness:** The custom vague term library and rule configurations, developed specifically for the AML domain, contribute significantly to the system's effectiveness.
- **Zero False Negatives Goal:** With FNR of 0.0606 at 70% threshold, the system rarely flags valid requirements as BAD, ensuring teams trust the quality gate.
- **Explainable AI:** The combination of rule-based explanations and ML confidence scores provides both accountability and intelligence.

10.2. Key Contributions

This research makes the following contributions:

- **A Curated Dataset:** A labeled dataset of 525 Jira tickets from the AML domain, balanced between good and semantically hollow requirements, with multi-dimensional categorization across 6 categories and 15 sub-areas.
- **A Custom Vague Term Library:** A domain-specific vocabulary of 10+ vague terms and phrases identified through analysis of historical agile tickets within the banking ecosystem.
- **A 10-Gate Rule Engine:** A deterministic rule engine with weighted checks covering structural completeness, template compliance, and domain-specific validation.
- **Comparative Model Evaluation:** Systematic evaluation of multiple ML classifiers with detailed confusion matrix analysis, achieving SVM (Linear) performance of 92.51% accuracy with zero false negatives.
- **A Hybrid Quality Scoring Framework:** A novel combination of deterministic rules and probabilistic ML classification that achieves 92.51% accuracy with balanced performance across all metrics.
- **Actionable Explainability**: The system provides specific reasons for quality flags, enabling teams to identify and address specific issues (e.g., "Missing User Story Format," "Due date is missing or invalid," "Semantic hollowness: Vague statement detected").
- **A Practical Implementation:** A working prototype with Streamlit dashboard and Ollama-based conversational interface, demonstrating real-world applicability.

Recommendation: Future iterations should focus on integrating this framework directly into the Jira webhook API to provide real-time, 'as-you-type' feedback to creators.

11. Abbreviations

Table 11 Abbreviations

Abbreviation	Full Form	Page No (first Occurrence)
AML	Anti-Money Laundering	4
API	Application Programming Interface	5
BITS	Birla Institute of Technology and Science	Front Cover
CSV	Comma Separated Values	4
DSE	Data Science and Engineering	Front Cover
F1-Score	Harmonic Mean of Precision and Recall	21
FNR	False Negative Rate	21
[redacted]	[redacted]	1
Jira	Agile Issue Tracking Platform	1
LLM	Large Language Model	3
LR	Logistic Regression	3
MTech	Master of Technology	Front Cover
NFR	Non-Functional Requirement	11
NLP	Natural Language Processing	6
RAG	Retrieval-Augmented Generation	38
RF	Random Forest Classifier	3
SVM	Support Vector Machine	3
TF-IDF	Term Frequency-Inverse Document Frequency	6
UI	User Interface	3
WILP	Work Integrated Learning Program	Front Cover
ML	Machine Learning	3
PO	Product Owner	10
JQL	Jira Query Language	18
RTE	Release Train Engineer	18
RBF	Radial Basis Function	12
SMOTE	Synthetic Minority Over-Sampling Technique	31
JSON	JavaScript Object Notation	46
AI	Artificial Intelligence	48

12. Literature References

The following references form the basis of this preliminary literature review:

[1] A. Ferrari et al., "Natural Language Processing (NLP) for Requirements Engineering: A Systematic Mapping Study," *ACM Computing Surveys*, 2021.

[2] W. Maalej, C. Montgomery, et al., "Mining Issue Trackers: Concepts and Techniques," *University of Hamburg*, 2024.

[3] D. Hallmann, K. Jacob, G. Lütgen, U. Schmid, and R. von der Weth, "USeR: A Web-based User Story eReviewer for Assisted Quality Optimizations," *Springer Nature*, 2025.

[4] M. Oruçević and A. Martini, "QualiTagger: Automating software quality detection in issue trackers," *Visma and University of Oslo (Preprint)*, 2024.

[5] O. Chaparro et al., "SPRINT: An Assistant for Issue Report Management," *College of William & Mary*, 2025.

[6] M. Nayebi et al., "Jira Mining for Agile Process Improvement," *IEEE Software*, 2020.

[7] F. Dalpiaz et al., "Requirements Quality: A Systematic Literature Review," *Requirements Engineering Journal*, 2019.

[8] T. Yue et al., "Automated Requirement Quality Assessment," *IEEE RE Conference*, 2022.

[9] Scikit-learn Developers, "Text Feature Extraction," *scikit-learn.org*, 2024. [Online].

[10] Streamlit Documentation, "Build Data Apps in Python," *streamlit.io*, 2024. [Online].

13. Checklist of Items for the Final Dissertation Report

This checklist is attached as the last page of the final report, duly completed and verified by the student.

S. No.	Item	Response
1	Is the final report neatly formatted with all the elements required for a technical report?	Yes
2	Is the Cover page in proper format as given in Annexure A?	Yes
3	Is the Title page (Inner cover page) in proper format?	Yes
4	(a) Is the Certificate from the Supervisor in proper format? (b) Has it been signed by the Supervisor?	Yes / Yes
5	Is the Abstract included in the report properly written within one page? Have the technical keywords been specified properly?	Yes / Yes
6	Is the title of the report appropriate — adequately descriptive, precise, and reflecting the scope of the actual work done?	Yes
7	Have you included the List of Abbreviations / Acronyms?	Yes
8	Does the report contain a summary of the literature survey?	Yes
9	Does the Table of Contents include page numbers; are pages, figures (captions at the bottom), tables (titles at the top) and appendices numbered properly with appropriate captions and titles?	Yes
10	Is the conclusion of the report based on discussion of the work?	Yes
11	Are References given at the end of the report, cited properly inside the text, and all cited in the body of the report?	Yes
12	Is the report format and content according to the guidelines (not a printout of a presentation or a user manual; source code excluded)?	Yes

Declaration by Student:

I certify that I have properly verified all the items in this checklist and ensure that the report is in proper format as specified in the course handout.

Place: [redacted]

Date: 25-July-2026

Name: Kamthekar Prasanna Yeshwantrao

ID No.: 2024DA04180